CLEARRUN · SRUJANA TECH FEST 2026 · F-03

System architecture

How the ClearRun cash flow app is built: what runs in the browser, what runs on the server, where data is stored, and how the outside services connect.

Version 5 · October 2026

At a glance

Front end

One page (`index.html`) in plain HTML, CSS and JavaScript. It has seven views: overview, cash flow, transactions, invoices, customer risk, settings and the demo.

Back end

A Node.js API (`lib/api.mjs`). On Vercel it runs as one serverless function, and on a laptop it runs through `server.mjs`. Both use the same code.

Data

Supabase Postgres, reached over its REST interface with a server-only key. Each business's workspace is stored as one JSON document, alongside the user, session and invite tables.

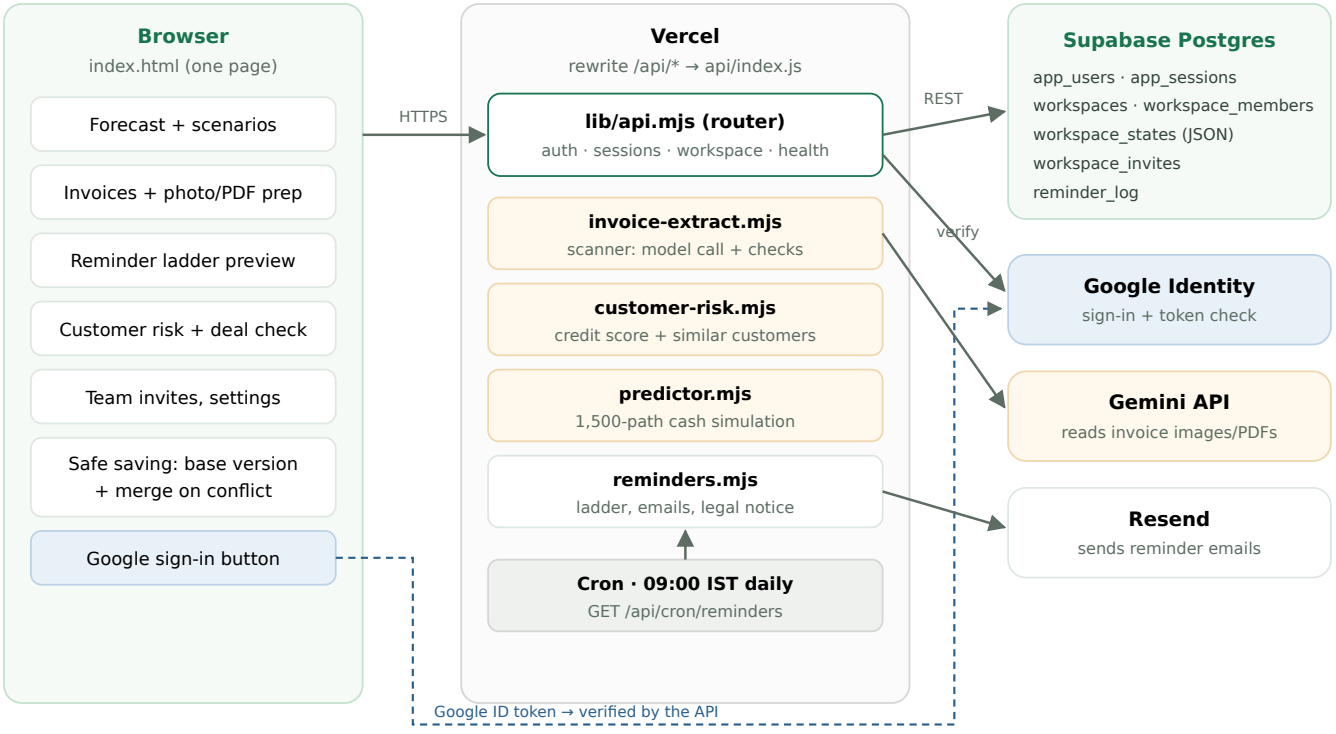


Figure 1. Components and connections. Amber boxes are the three models described in the "Models" document.

Why it is built this way

Decision	Reason
One HTML page, no framework	Loads fast on a phone and needs no build step, so a hackathon team can edit it directly.
One serverless function for every API path	Vercel only supports catch-all route files inside Next.js. A single rewrite to `api/index.js` works for every path and stays within the free plan's function limits.
Workspace stored as one JSON document	Cash, cash events, invoices, reminder settings and customer profiles change together and are always loaded together. One document keeps reads simple and lets teammates share a workspace.
Models run inside the API, not in Python	The customer risk model was written in Python and rebuilt in JavaScript with identical rules, so it needs no second runtime. It was checked against the Python version: all 18 test customers match.
AI only where it is needed	Only invoice reading uses an AI model. Scores, forecasts and reminders are plain arithmetic and rules that can be explained and repeated exactly.

Key request flows

Sign in with Google

1. The browser asks `/api/config` for the Google client ID. It is stored only in the server settings, so the browser and the server always use the same one.
2. Google's button returns a signed ID token. The browser sends it to `POST /api/auth/google`.
3. The server has Google verify the token. It checks that the token was issued for ClearRun's client ID, that the email is verified, and that the token has not expired.
4. If the Google account is new, the server creates a user and a private workspace for it. It then starts a session.

Phone and password sign in

Passwords are stored as salted **scrypt** hashes, never as plain text. Usernames are matched exactly, ignoring case. Characters that the database would treat as wildcards are escaped, so one person cannot match another's account. Repeated failed attempts from one visitor are slowed down.

Sessions

A session is a random 32-byte token kept in an `HttpOnly`, `SameSite=Lax`, `Secure` cookie. The database keeps only a SHA-256 hash of the token. Sessions last a year and are extended at most once a day, which avoids a database write every time the dashboard refreshes.

Saving without overwriting a teammate

> Each save sends the version of the workspace it started from. The server updates the record only if that version is still current, in one atomic step. If a teammate saved first, the server sends back their version. The browser then merges the two by item (cash events, invoices, customers), keeping both people's changes, and saves again.

Invoice scan

The browser straightens, resizes and brightens a photo, or sends a PDF as it is. The server sends it to Gemini, checks every value it returns, reads the invoice a second time if something does not add up,

and returns the fields with a confidence for each. The user checks the fields and saves. Details are in the Models document.

Daily reminders

Vercel Cron calls `/api/cron/reminders` at 09:00 IST. For each invoice with reminders turned on, the job finds the latest step of the ladder that has been reached and sends that email through Resend. It first writes a row to `reminder_log`, which allows only one row per invoice and step, so the same email can never be sent twice. A test address setting sends every reminder to the owner instead of customers, and a separate switch must be turned on before real customers are emailed.

Security

Area	Measure
Secrets	The Supabase service key, Gemini key and Resend key are stored only in server settings. None of them are sent to the browser.
Database access	Row-level security is turned on, and the anonymous and signed-in roles have no rights. Only the server's key can read or write data.
Cross-site requests	Write requests are refused unless they come from the app's own address.
Abuse limits	10 failed sign-ins and 20 sign-ups per visitor per minute, and 8 invoice scans per user per 10 minutes.
Uploaded content	Text in invoices is treated as data. The scanner is told to ignore anything that looks like instructions, and all output is escaped before it is shown.

Configuration and checks

Setting	Purpose
`SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`	Database connection (required)
`GOOGLE_CLIENT_ID`	Google sign-in. Several client IDs can be listed, separated by commas.
`GEMINI_API_KEY`, `GEMINI_MODEL`	Invoice scanning. The model setting can list several models in fallback order.
`RESEND_*`, `CRON_SECRET`, `REMINDER_*`	Reminder emails, the daily job, test mode and the live switch

`GET /api/health` reports which settings are present and whether the database can be reached. It explains plain causes such as a wrong Supabase URL or missing tables, without revealing any secret.

Testing

Automated checks run before each release: 63 browser and API checks, 13 Google sign-in and routing checks, 11 scanner checks, 12 reminder checks, and the risk model compared with the Python original on 18 customers. The tests use a stand-in database and simulated responses from Google and Gemini, so the live site was checked separately.